

Ver el error de hoy y acordar qué construimos

Un agente que mapea el catálogo a Shopee, decidido paso a paso.

War Room · Alephee × Craftech × AWS · 1/10/2026

Ver el error de hoy y acordar qué construimos

El código está en el repositorio.

<https://github.com/craftech-io/alephee-poc-catalogo>

- Repositorio: <https://github.com/craftech-io/alephee-poc-catalogo>
- El agente: `core/src/catalog/`, con `v1.py` y `v2.py` como punto de entrada
- Versiones: la rama `main` y los tags `reinicio-v1` y `reinicio-v2`
- Los grafos de los Workflows: `docs/workflows.md`, generados desde el código

Ver el error de hoy y acordar qué construimos

Lo que vamos a hacer hoy.

Decidimos, construimos y medimos. Cada bloque cierra con una versión que funciona.

- 1 · PUNTO DE PARTIDA | El error de hoy, el alcance y el contrato
- 2 · DISEÑAR EL AGENTE | Tipo de aplicación, prompt, dónde corre, modelo, stack y criterio de éxito
- 3 · V1 · EL AGENTE RESPONDE | Una sola llamada al modelo, sin herramientas: qué resuelve solo y qué no
- 4 · V2 · TABLAS Y CONTROL | Las tablas en código, un agente que decide valores, validación, correcciones y caché
- 5 · LA PRUEBA | El proceso de hoy contra la V1 y la V2, y el camino a producción

Ver el error de hoy y acordar qué construimos

De un producto de Alephee a una publicación de Shopee.

Caso real · SKU 88904447 · Correia dentada · catálogo GM Brasil (Mercado Libre Brasil) → Shopee Brasil

Entra (Alephee)	Sale (Shopee)	Quién lo resuelve
Categoría: Correias Dentadas	Distribuição e Correias	La tabla
Condición del ítem: Novo	Condition: New	La tabla y la lista del canal
Tiempo de garantía: 6 meses	Duração da Garantia: 6 Months	El agente
Código OEM: no viene	Faltante, con motivo (hoy: ABS Plastic, rechazado por Shopee)	El guardrail

Ver el error de hoy y acordar qué construimos

Hoy: dos llamadas, un merge y un presupuesto que se agota.

- Categoría: la tabla `reference_category` y, si no alcanza, el modelo elige
- Atributos: dos llamadas en paralelo (con y sin referencia) y un merge por código
- USD 350 por mes: cuando se agota, se publica sin atributos

Ver el error de hoy y acordar qué construimos

Lo que Shopee rechazó y lo que aceptó mal.

Datos reales · WarRoom.zip del 24/09 · 30 productos

Caso	Qué salió	Qué pasó
88904447	Código OEM = ABS Plastic	Rechazado: el valor no pertenece al atributo
93221445	Inmetro Certification con varios valores	Rechazado: solo admite un valor
9 de 30	Valor -1 publicado como valor	Aceptado por Shopee
98550368	14 de 15 URN sin sufijo del canal	Aceptado por Shopee

Ver el error de hoy y acordar qué construimos

¿Qué hace y qué no hace?

DECISIÓN 1

¿Qué alcance tiene el agente del war room?

A · Un canal (Shopee) y una familia de productos: categoría + atributos

B · Todos los canales conectados (Shopee, Magalu, Tienda Nube...)

C · También título, descripción y marca

PROPUESTA · A. Un canal y una familia alcanzan para decidir el método y medirlo. Marca, título y descripción son redacción libre y no mapeo, así que quedan fuera. Los otros canales repiten el mismo diseño con sus propias tablas.

decisiones/01-alcance.md

Ver el error de hoy y acordar qué construimos

Entra un producto. Sale una propuesta de publicación.

- Entrada: el producto de Alephee (SKU, nombre, descripción, categoría legacy, atributos)
- Salida: la categoría de Shopee más los atributos con URN, valueId y valor
- Y dos listas más: missing (obligatorios sin dato) y rejected (atributos descartados, con motivo)

Ver el error de hoy y acordar qué construimos

El contrato, en código.

core/src/catalog/models.py · líneas 38–53

```
38 # A product attribute that was discarded, with the reason. Rejections added by the guardrails
39 # carry the channel urn here instead, because the validator does not know the source attribute.
40 class RejectedAttribute(_Strict):
41     legacyId: str = Field(description="Product attribute id, without the 'urn:attribute:' prefix.")
42     reason: str
43
44
45 # The full answer for one product. In V2 the category is fixed by `reference_category` and the
46 # guardrails force it back if the model returns a different one.
47 class Listing(_Strict):
48     """Result of mapping one product to a channel listing."""
49
50     category: str | None = Field(description="Channel category URN, copied exactly. Null if it cannot be resolved.")
51     attributes: list[MappedAttribute]
52     missing: list[MissingAttribute] = Field(description="Mandatory channel attributes (or the category) that could not be filled.")
53     rejected: list[RejectedAttribute] = Field(description="Product attributes that are discarded, with the reason.")
```

Listing es lo que entrega la V1. Si el modelo devuelve algo que no cumple este esquema, el Workflow informa el error.

Ver el error de hoy y acordar qué construimos

¿Qué recibe y qué entrega?

DECISIÓN 2

¿Cómo se informa lo que no se pudo mapear?

A · No se informa: se publica lo que hay (hoy)

B · missing y rejected con motivo, en la misma salida

C · Un archivo de log aparte que alguien revisa

PROPUESTA · B. El faltante viaja con la publicación, y quien revisa ve el producto, el atributo y el motivo juntos. Un log aparte se separa del dato y nadie lo mira. La decisión 12 define qué pasa después con esas listas.

decisiones/02-contrato.md

Tomar las decisiones que definen la V1

Diseñar antes de escribir.

Siete decisiones que definen la V1

Tomar las decisiones que definen la V1

Cuatro formas de usar un modelo.

- SINGLE PROMPT | Una llamada y una respuesta. Barato y fácil de evaluar, pero no consulta nada
- WORKFLOW | Pasos fijos en código, y el modelo participa en uno o dos. Es predecible
- AGENTE | El modelo decide qué herramienta pedir y cuándo entregar, con un límite de rondas
- MULTIAGENTE | Varios agentes con roles distintos. Solo cuando uno no alcanza

Tomar las decisiones que definen la V1

¿Single prompt o agente?

La V1 se construye con la opción A · si gana otra, vemos qué cambia

DECISIÓN 3

¿Con qué tipo de aplicación empezamos?

A · Una sola llamada estructurada (V1) y sumar herramientas solo cuando un fallo lo justifique

B · Agente con herramientas desde el inicio

C · Workflow determinista, sin modelo

PROPUESTA · A. La V1 muestra qué resuelve el modelo solo y qué no, y cada fallo justifica la capa siguiente. También es la buena práctica de LlamaIndex: si el modelo no tiene herramientas que elegir, no hace falta un agente. El FunctionAgent entra en la V2. C no alcanza, porque elegir el valor equivalente de una lista (por ejemplo, '1' es Sim) necesita interpretación.

decisiones/03-tipo-de-aplicacion.md

Tomar las decisiones que definen la V1

Un prompt tiene cinco partes.

01 Rol: quién es

02 Tarea: qué hace y con qué entrada

03 Reglas: qué nunca hace

04 Formato: cómo entrega

05 Cuando falta dato: qué hace

Tomar las decisiones que definen la V1

Buenas prácticas que cambian el resultado.

- Lo fijo primero y lo variable al final: así funciona la caché de prompt (F4, F7)
- Salida con esquema, en lugar de pedir JSON en el texto
- Decir qué hacer cuando no sabe, además de qué hacer

Tomar las decisiones que definen la V1

¿Cómo garantizamos el formato?

La V1 usa la opción B · si gana otra, vemos qué cambia

DECISIÓN 4

¿Cómo se garantiza que la salida cumpla el contrato?

- A · Pedir JSON en el texto y parsearlo (hoy)
- B · Salida estructurada con un modelo Pydantic: el modelo devuelve un Listing validado
- C · Corregir la respuesta después con expresiones regulares

PROPUESTA · B. El formato deja de ser un pedido en el prompt y pasa a ser un contrato. En LlamaIndex es `as_structured_llm(Listing)`: el modelo responde por tool calling y la librería valida contra el esquema. Si la salida no cumple, el Workflow devuelve el error en lugar de publicar algo a medias. C arregla síntomas: si el modelo omite un campo, la expresión regular no lo inventa.

decisiones/04-salida-estructurada.md

Tomar las decisiones que definen la V1

Salida estructurada en LlamaIndex.

core/src/catalog/v1.py · líneas 84–98

```
84 @step
85 async def map(self, ev: ContextReady) -> MappingCompleted:
86     """Call the model with `Listing` as the output schema and return what it produced."""
87     try:
88         # `as_structured_llm` sends `Listing` as the schema and parses the answer into it;
89         # an answer that does not fit the schema raises here instead of leaking bad JSON.
90         response = await self.llm.as_structured_llm(Listing).achat(ev.messages)
91     except Exception as exc: # noqa: BLE001 — any failure becomes a reported error, except credentials
92         # Expired credentials would fail every remaining product: let them stop the run.
93         if is_auth_error(exc):
94             raise
95         return MappingCompleted(listing=None, error=f"{type(exc).__name__}: {exc}")
96     # `raw` holds the parsed `Listing` instance. V1 has no guardrails: the listing is returned
97     # as the model wrote it, which is what the V1 column measures.
98     return MappingCompleted(listing=response.raw)
```

Una sola llamada. response.raw ya es un Listing validado.

Tomar las decisiones que definen la V1

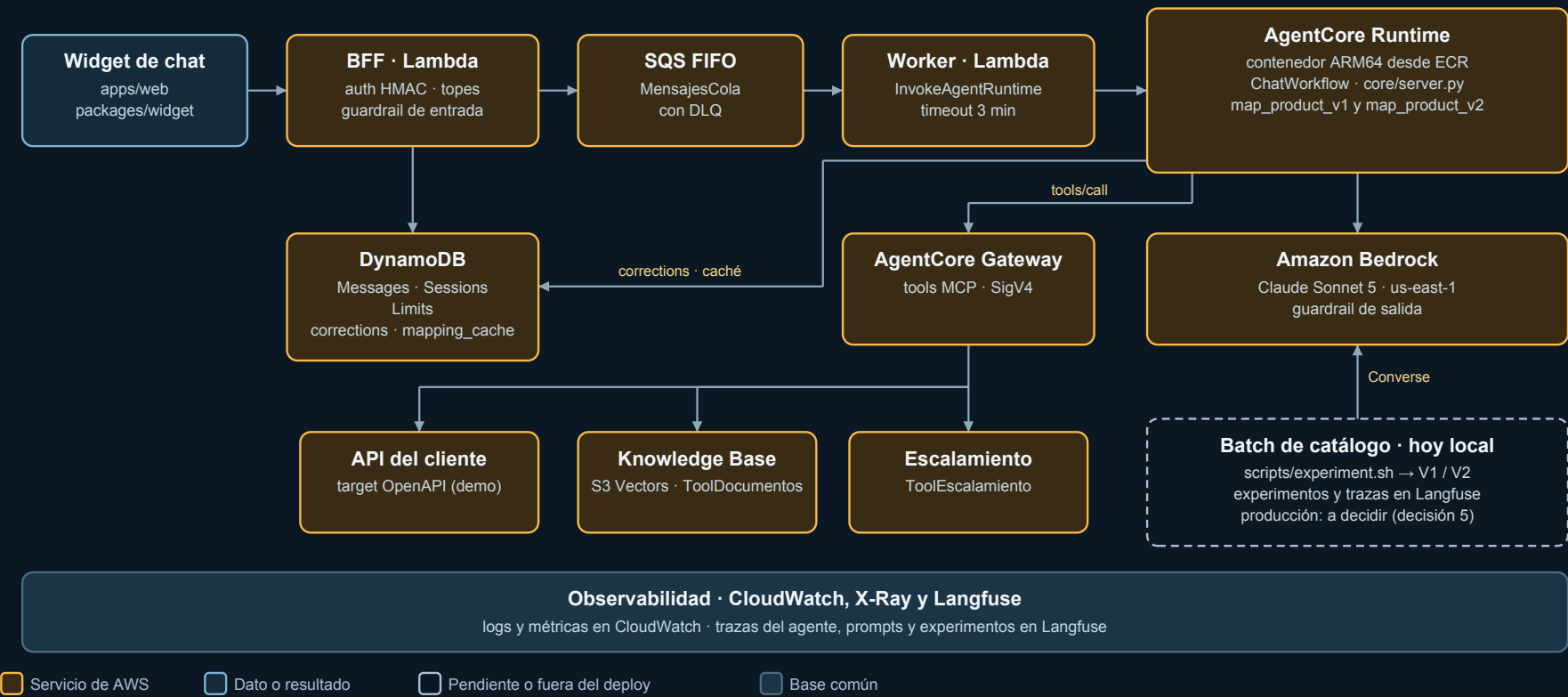
Dónde corre: AgentCore o contenedor propio.

Fuentes F1 a F3b en docs/warroom/fuentes.md

- AGENTCORE RESUELVE | Runtime sin servidores, identidad, memoria, gateway de herramientas y observabilidad. Son servicios separados que se pueden usar juntos o no (F2)
- DUDAS | No hay cifra publicada de arranque en frío. Las sesiones terminan a los 15 minutos sin actividad y duran 8 horas como máximo (F3b). Se cobra por CPU y memoria de la sesión (F3), y hay que confirmar la región
- ALTERNATIVA | Contenedor o Lambda propios: más control y más trabajo de operación. El agente es el mismo

Tomar las decisiones que definen la V1

La infraestructura, de punta a punta.



Es la infraestructura del template en infra/sst. El chat se despliega en la cuenta sandbox con el stage warroom, y las trazas del agente van a Langfuse. AgentCore Memory se sacó del stack.

Tomar las decisiones que definen la V1

¿Dónde corre?

DECISIÓN 5

¿Dónde corre el agente?

A · Hoy en local y el chat de demo en AgentCore (stage warroom). El batch como proceso de Alephee se decide con el camino a producción

B · Todo en AgentCore desde el inicio

C · Todo en la infraestructura actual de Alephee

PROPUESTA · A. Hoy construimos y medimos en local. El chat va a AgentCore porque es su caso natural. Para el batch hay que medir sesiones, costo y arranque antes de decidir, y lo retomamos a las 16:15.

decisiones/05-donde-corre.md

Tomar las decisiones que definen la V1

Elegir el modelo: qué pesa en este caso.

- Seguir reglas sin inventar valores: es lo que más falla hoy
- Costo por token con caché de prompt: el presupuesto actual es de USD 350 por mes
- Latencia: no es una restricción (hoy el batch tarda 13 s por producto)
- Que esté disponible en la región y habilitado en la cuenta

Tomar las decisiones que definen la V1

Qué hay en Bedrock.

Fuente F6 · Models at a glance · verificado el 30/09/2026

Proveedor	Familias	Para este caso
Anthropic	Claude 5.x (Sonnet 5.5, Opus 5.5, Fable 5.1, Sonnet 5...), 4.x (Haiku 4.5...), 3.x	Candidato principal: reglas y tool calling. Sonnet 5 elegido el 25/09; 5.5 y Haiku 4.5 a comparar
Amazon	Nova 2 Lite, Nova Premier, Pro, Lite, Micro	Alternativa de costo a medir
OpenAI	GPT-5.x, GPT-6, GPT OSS	Continuidad con el proveedor actual, dentro de AWS
Meta · Mistral · DeepSeek · Qwen · otros	Llama 3.x y 4, Mistral Large 3, DeepSeek V3.2, Qwen3...	Abiertos; evaluar si el costo lo justifica

Tomar las decisiones que definen la V1

¿Qué modelo?

Tomada el 25/09 (CLAUDE.md, decisión 6) · hoy se valida o se cambia

DECISIÓN 6

¿Con qué modelo construimos y medimos?

A · Claude Sonnet 5 en Bedrock, vía Converse

B · Un modelo más chico y barato (Claude Haiku 4.5) y medir

C · Seguir con OpenAI y solo reordenar el prompt para la caché

PROPUESTA · A para construir hoy. Es la decisión del 25/09 y el ID está verificado en Converse. Claude Sonnet 5.5 también está en Bedrock (F6). Compararlo con Haiku 4.5 (B) es una corrida más del experimento con otra variable de entorno, y queda como pendiente con dueño. C mejora el costo pero no resuelve el contrato ni el control.

decisiones/06-modelo.md

Tomar las decisiones que definen la V1

El único módulo que sabe de Bedrock.

core/src/catalog/llm.py · líneas 12–26

```
12 # Converse accepts the `us.` and `global.` profiles; the bare model id has no on-demand throughput.
13 DEFAULT_MODEL_ID = "us.anthropic.claude-sonnet-5"
14 _AUTH_ERRORS = (NoCredentialsError, SSOError, TokenRetrievalError, UnauthorizedSSOTokenError)
15
16
17 def create_llm(env=os.environ) -> BedrockConverse:
18     """Build the Bedrock Converse client from the environment (MODEL_ID, AWS_REGION, AWS_PROFILE)."""
19     return BedrockConverse(
20         model=env.get("MODEL_ID", DEFAULT_MODEL_ID),
21         region_name=env.get("AWS_REGION", "us-east-1"),
22         # Local runs use the SSO profile; inside the Runtime boto3 takes the role.
23         profile_name=env.get("AWS_PROFILE") or None,
24         # High ceiling on purpose: an answer cut off at the limit is not a valid Listing.
25         max_tokens=16000,
26     )
```

En local usa el perfil SSO; en el Runtime de AgentCore toma el rol de la cuenta.

Tomar las decisiones que definen la V1

Stack y harness.

- Python + LlamaIndex Workflows: pasos y eventos tipados, y cada paso se prueba por separado
- BedrockConverse: salida estructurada, tool calling y caché de prompt sin código propio
- Langfuse: el prompt versionado, las trazas de cada llamada y los experimentos
- Harness de desarrollo: propone cambios en el código, y Gastón los revisa y corre los tests

Tomar las decisiones que definen la V1

¿Con qué lo construimos?

Tomada el 28/09 · Langfuse se sumó el 1/10 · hoy se valida

DECISIÓN 7

¿Qué stack usamos para el agente?

A · Python + LlamaIndex Workflows + BedrockConverse, con Langfuse para prompts y evaluación

B · Llamadas directas al SDK, sin framework (como hoy)

C · Otro framework de agentes (Strands, LangGraph...)

PROPUESTA · A. Es el stack del template que Crafttech ya opera. El Workflow no conoce Bedrock y cada paso se prueba con dobles. B repite el código de loop, herramientas y caché que un framework ya trae. C es válido, y elegimos A por continuidad con lo que ya está probado.

decisiones/07-stack.md

Tomar las decisiones que definen la V1

¿Cuándo está bien hecho?

Esquema de Shopee y salida esperada MOCK · validar con el equipo de catálogo

- Exacto: categoría correcta, ningún atributo de más ni de menos, nada fuera de la lista del canal, sin duplicados y con los faltantes informados
- También medimos precisión y recall de atributos, valores inválidos, tokens y segundos, todo en Langfuse
- Dataset: 30 productos reales con su publicación actual. El esquema de Shopee y la salida esperada son simulados

Tomar las decisiones que definen la V1

La métrica, en código.

core/src/catalog/evaluation.py · líneas 25–42

```
25 class CaseResult:
26     """Score of one case: attribute counts (true/false positives, false negatives) and the
27     urns behind each kind of error, so a Langfuse comment can name them."""
28
29     category_ok: bool
30     tp: int
31     fp: int
32     fn: int
33     invalid: list[str] = field(default_factory=list)
34     duplicates: list[str] = field(default_factory=list)
35     missing_not_detected: list[str] = field(default_factory=list)
36     extra_missing: list[str] = field(default_factory=list)
37
38     @property
39     def exact(self) -> bool:
40         """All checks pass at once: the strictest metric of the table."""
41         return (self.category_ok and self.fp == 0 and self.fn == 0 and not self.invalid
42                 and not self.duplicates and not self.missing_not_detected and not self.extra_missing)
```

La misma función mide el proceso de hoy y cada versión.

Tomar las decisiones que definen la V1

¿Cuál es el número que aceptamos?

DECISIÓN 8

¿Qué criterio de éxito acordamos para la prueba de las 16:15?

A · Cero valores inválidos, cero duplicados y todos los obligatorios informados en los 30. Los exactos se informan, pero no son condición hasta validar la salida esperada con catálogo

B · Exactos iguales o mayores al 80 %

C · Solo precisión y recall de atributos

PROPUESTA · A. Con una salida esperada mock, el exacto castiga aciertos que hoy nadie mapea: el proceso actual saca 7 de 30. Los inválidos, duplicados y obligatorios sin informar sí son errores seguros, y hoy hay 47, 5 y 2. El número final se fija en la sala.

decisiones/08-criterio-de-exito.md

Tomar las decisiones que definen la V1

¿Con qué dataset?

DECISIÓN 9

¿Sobre qué datos construimos y medimos?

A · Los 30 reales del zip, con esquema y expected mock rotulados

B · Solo los 10 casos mock de borde

C · Esperar el esquema oficial de Shopee para empezar

PROPUESTA · A. Son productos reales, con publicaciones reales y rechazos reales de Shopee. Lo simulado queda rotulado y se valida con catálogo después. Los 10 casos mock quedan como pruebas de borde.

decisiones/09-dataset.md

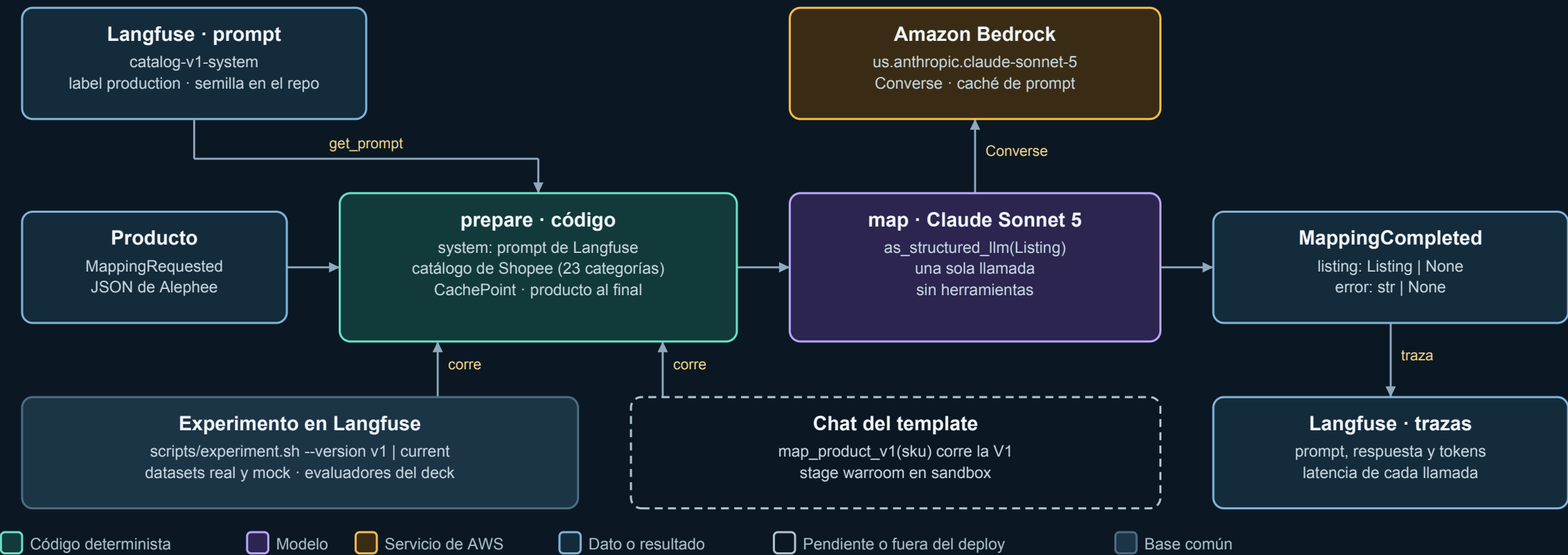
Construir, correr y leer la primera versión

V1 · el agente responde.

Una sola llamada al modelo, sin herramientas. Esperamos que falle, y esos fallos justifican lo que sigue.

Construir, correr y leer la primera versión

La V1 por dentro.



La V1 es un Workflow de dos steps. El experimento y el chat corren el mismo Workflow, y Langfuse guarda el prompt, las trazas y los resultados.

Construir, correr y leer la primera versión

El prompt de sistema.

core/src/catalog/prompts/catalog-v1-system.txt · líneas 7–18

```
7 Rules:
8 - Copy the category URN and every attribute URN exactly from the Shopee catalog. Never invent a URN.
9 - If the channel attribute has a list of values, choose the equivalent value from that list and use
10   its id and its name exactly as listed. If no value is equivalent, do not fill it.
11 - If the channel attribute is free text, use valueId "0" and the product value as it is.
12 - Never translate values: Shopee values stay in Portuguese, exactly as the catalog lists them.
13 - Never invent values. Use only data present in the product attributes.
14 - The values "-1", "N/A" or empty mean there is no data.
15 - If a mandatory channel attribute cannot be filled, add it to "missing" with the reason.
16 - If a product attribute cannot be used, add it to "rejected" with its legacyId and the reason.
17 - Each channel attribute appears at most once.
18 - If you cannot choose a category, return category null and add {"urn": "category"} to "missing".
```

Esta copia es la semilla. La versión que se usa vive en Langfuse con el label production.

Construir, correr y leer la primera versión

El prompt vive en Langfuse.

core/src/catalog/prompts.py · líneas 31–39

```
31
32 def get_system_prompt(name: str, client) -> SystemPrompt:
33     """The `production` prompt from Langfuse, or the seed when there is no client or Langfuse fails."""
34     seed = load_seed(name)
35     if client is None:
36         return SystemPrompt(seed, name, None)
37     # With `fallback`, the SDK returns the seed instead of raising when Langfuse is unreachable,
38     # and flags it with `is_fallback`, so the version is reported as None (the seed), not a number.
39     prompt = client.get_prompt(name, label=LABEL, fallback=seed)
```

version None quiere decir que se usó la semilla.

Construir, correr y leer la primera versión

Lo fijo primero, el producto al final.

core/src/catalog/v1.py · líneas 53–67

```
53 def build_messages(system_prompt: str, schemas: dict[str, dict], product: dict) -> list[ChatMessage]:
54     """System prompt, then the static catalog, then the product.
55
56     Today's prompts put the product first and the big lists after it, which defeats prompt
57     caching. Here the order is reversed: everything static comes first.
58     """
59     return [
60         ChatMessage(role="system", content=system_prompt),
61         ChatMessage(role="user", blocks=[
62             TextBlock(text=catalog_text(schemas)),
63             # Everything before this point is identical for every product: Bedrock caches it.
64             CachePoint(cache_control=CacheControl(type="default")),
65             TextBlock(text=product_text(product)),
66         ]),
67     ]
```

En la traza de Langfuse se ven los tokens leídos de caché.

Construir, correr y leer la primera versión

El Workflow cierra con un evento tipado.

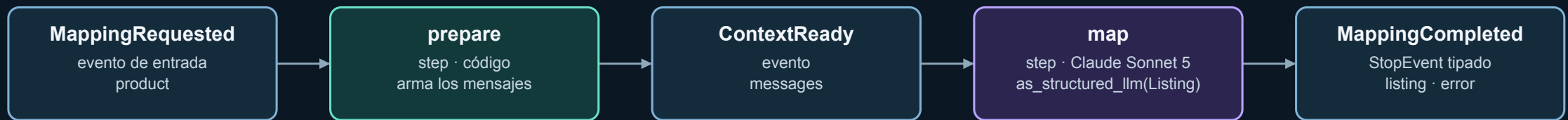
core/src/catalog/events.py · líneas 22–38

```
22
23 class MappingRequested(StartEvent):
24     """Start of both workflows: `workflow.run(product=...)` builds this event from the kwargs."""
25
26     product: dict
27
28
29 class ContextReady(Event):
30     """V1: the messages for the single structured call, already in cache-friendly order."""
31
32     messages: list[ChatMessage]
33
34
35 class MappingCompleted(StopEvent):
36     """End of a mapping. No field is called `result`: it would collide with StopEvent's."""
37
38     listing: Listing | None
```

El batch, el chat y los tests leen fin.listing sin parsear nada.

Construir, correr y leer la primera versión

El flujo de la V1, según LlamaIndex.



■ Código determinista ■ Modelo ■ Dato o resultado

Este grafo lo dibuja LlamaIndex a partir de las firmas de los steps: cada step recibe un evento y devuelve otro.

Construir, correr y leer la primera versión

Demo · la V1 sobre un producto real.

```
scripts/experiment.sh --version v1 --data mock --case 01-real-calota-aro14
```

- La categoría que eligió el modelo y la esperada
- Cada atributo: de dónde sale el valor y si está en la lista del canal
- missing y rejected: si informa lo que no pudo o completa igual
- En Langfuse: la traza con el prompt, la respuesta, los tokens y los segundos, y los scores del caso

Respaldo · el experimento v1-mock que corrimos antes de la sesión, abierto en Langfuse

Construir, correr y leer la primera versión

Qué falló en la V1 y qué capa lo resuelve.

Medido el 1/10 en Langfuse · V1 sobre los 30 reales · salida esperada MOCK

Qué medimos	V1 · 30 reales	Qué capa lo ataca en la V2
Casos exactos	3 de 30 (hoy: 7)	Se compara en la prueba
Categoría correcta	28 de 30, optimista: elige entre 23 que incluyen la correcta	La categoría sale de reference_category, en código
Valores inválidos	3 (hoy: 47)	validate en cada entrega y clean al final
Duplicados	0 (hoy: 5)	validate y clean
Recall de atributos	0,57: faltan 4 de cada 10 atributos esperados	Los campos de reference_attribute en código. Medido el 1/10: 0,60 en la V2

Resolver con las tablas lo que saben y controlar lo que decide el agente

V2 · tablas y control.

El código resuelve lo que saben las tablas. El agente decide el resto y entrega por una herramienta que valida.

Resolver con las tablas lo que saben y controlar lo que decide el agente

Una herramienta es una función que el modelo pide y el código ejecuta.

01 El modelo pide `lookup_corrections(attribute_urn, product_value)`

02 El programa ejecuta la función: busca en la tabla `corrections` de DynamoDB

03 Devuelve `{correction: {valueId, value}}` o `{correction: null}`

04 El modelo sigue con ese dato, no con su memoria

Resolver con las tablas lo que saben y controlar lo que decide el agente

¿Qué decide la tabla y qué decide el agente?

La V2 se construyó con la opción A · si gana otra, vemos qué cambia

DECISIÓN 10

¿Qué decide la tabla y qué decide el agente?

A · La tabla fija la categoría y los campos; el agente solo elige el valor equivalente de la lista y cubre lo que la tabla no tiene

B · El agente puede corregir la tabla si cree que está mal

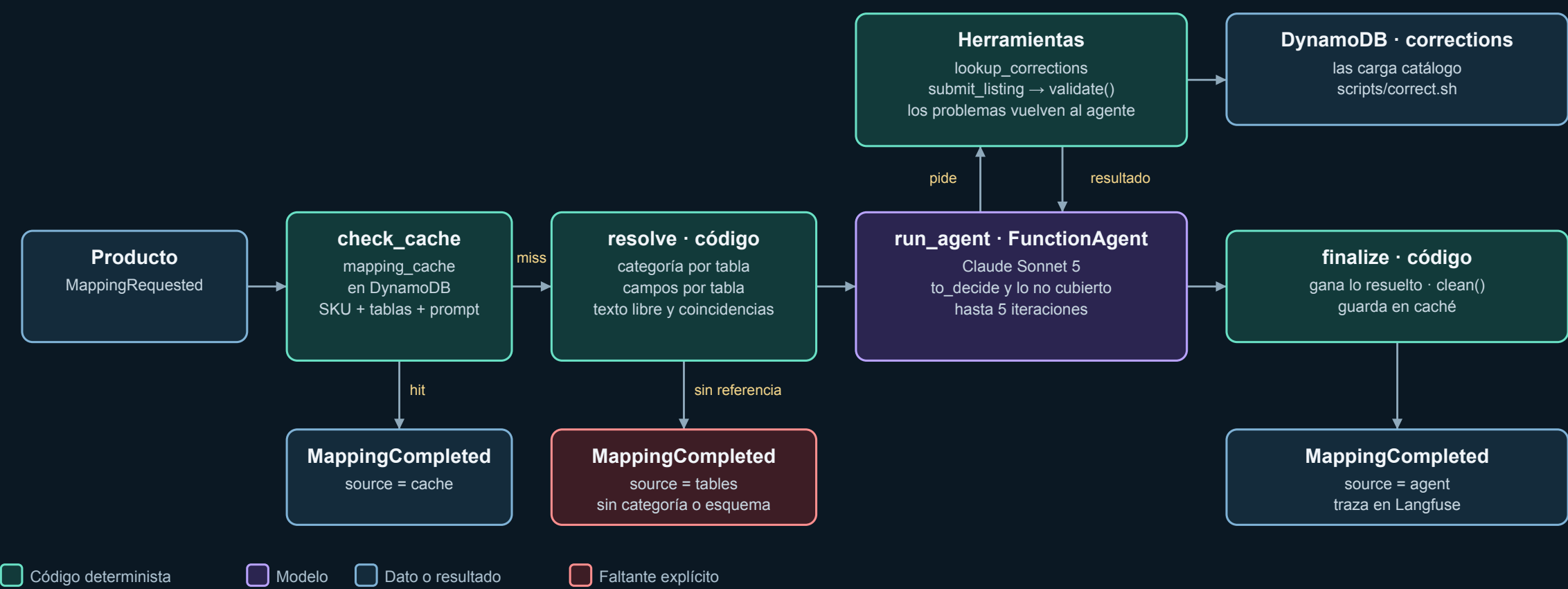
C · Todo por tabla; sin modelo

PROPUESTA · A. La tabla manda, como se acordó el 25/08 con Juan David. En la V2, el step resolve fija en código la categoría con `reference_category` y los campos con `reference_attribute`. Si el campo es texto libre o el valor coincide con la lista del canal, también lo resuelve el código. El agente recibe solo lo que queda: los valores de lista por decidir y los atributos que la tabla no cubre. Si cambia algo resuelto, gana lo resuelto.

decisiones/10-tabla-vs-agente.md

Resolver con las tablas lo que saben y controlar lo que decide el agente

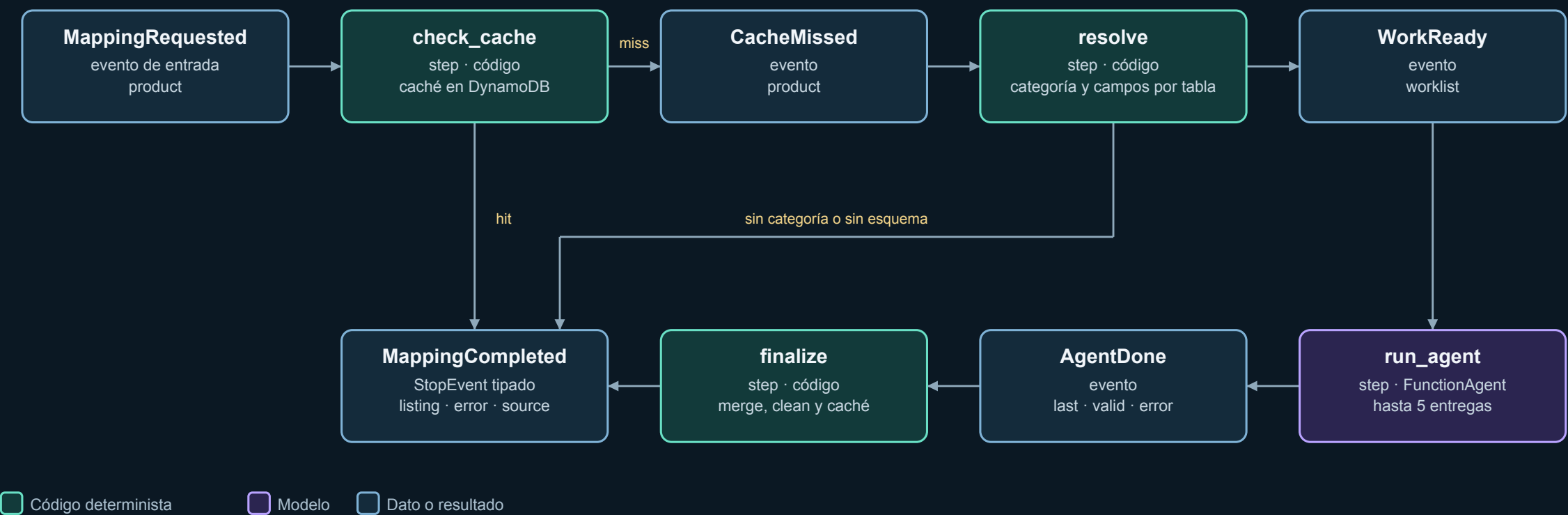
La V2 por dentro.



Verde es código y violeta es el modelo. El código resuelve lo que saben las tablas; el agente decide valores y entrega por una herramienta que valida.

Resolver con las tablas lo que saben y controlar lo que decide el agente

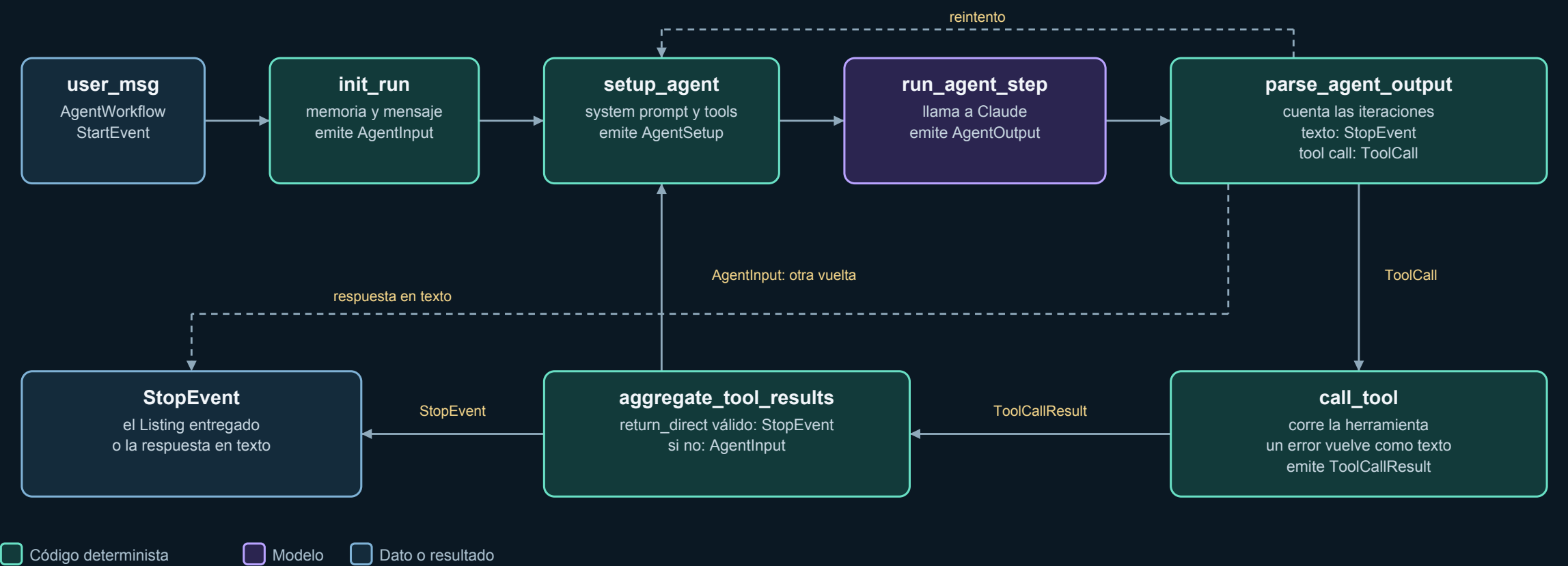
El flujo de la V2, según LlamaIndex.



La V2 tiene tres salidas: desde la caché, desde las tablas sin llamar al modelo, o después del agente.

Resolver con las tablas lo que saben y controlar lo que decide el agente

Adentro de run_agent: el ciclo del FunctionAgent.



El ciclo propio del FunctionAgent, leído de la librería. submit_listing es return_direct: solo una entrega válida corta el ciclo.

Resolver con las tablas lo que saben y controlar lo que decide el agente

Las dos herramientas del agente.

El agente solo recibe lo que falta decidir: `to_decide`, `unmapped_product` y `uncovered_channel`

Herramienta	Qué recibe	Qué devuelve	Para qué sirve
<code>lookup_corrections</code>	<code>attribute_urn</code> y <code>product_value</code>	La corrección del equipo de catálogo, o <code>correction: null</code>	Ver si catálogo ya corrigió ese valor. La corrección manda sobre el modelo
<code>submit_listing</code>	El Listing completo: <code>category</code> , <code>attributes</code> , <code>missing</code> y <code>rejected</code>	Si pasa la validación, el Listing y fin del ciclo. Si no, la lista de problemas	Entregar. Lo valida el código, no el modelo

Resolver con las tablas lo que saben y controlar lo que decide el agente

Un caso real: pastillas de freno, SKU 98550735.

Datos reales · corrida de la V2 del 1/10, leída de Langfuse · el esquema de Shopee es MOCK

Paso	Quién	Qué pasó
Hoy	Proceso actual	Shopee rechazó la publicación: faltaba Inmetro Certification, que es obligatorio
resolve	Código	Categoría Pastilha de Freio por reference_category. Cuatro campos sin modelo: Novo, Brasil, Inmetro 007077/2015 y Modelo
lookup_corrections × 3	Agente	lookup_corrections(101826, "1"), (100134, "Metal") y (100037, "Brasil"): las tres devuelven correction: null
submit_listing	Agente, validado por el código	10 atributos: "1" pasa a Sim, 4 pastillas, Dianteira. Pasa la validación al primer intento
finalize	Código	Sale el Listing con Inmetro incluido, 17 atributos rechazados con motivo y 0 faltantes. Cuatro llamadas al modelo

Resolver con las tablas lo que saben y controlar lo que decide el agente

Lo que resuelve el código.

core/src/catalog/worklist.py · líneas 62–79

```
62 # First product attribute wins a target: two legacy fields pointing to the same Shopee
63 # attribute must not produce a duplicate (the kind of duplicate today's output shows).
64 if target in covered:
65     continue
66 covered.add(target)
67 definition = definitions[target]
68 values = _values(definition)
69 unit = str(attribute.get("unit") or "").strip()
70 # Free text with no unit: the product value is copied as is. With a unit it goes to the
71 # agent, which decides whether and how the unit fits the channel.
72 if not values and unit in NO_DATA:
73     resolved.append(MappedAttribute(urn=target, valueId="0", value=value, unit=None))
74     continue
75 # List attribute whose value matches a channel value by name: take the channel's id and
76 # spelling. Anything else needs judgment and goes to the agent.
77 match = next((v for v in values if normalize(v["name"]) == normalize(value)), None)
78 if match is not None:
79     resolved.append(MappedAttribute(urn=target, valueId=match["id"], value=match["name"], unit=None))
```

Texto libre y coincidencia exacta salen sin modelo. Lo que no coincide va a `to_decide`.

Resolver con las tablas lo que saben y controlar lo que decide el agente

La entrega pasa por una herramienta que valida.

core/src/catalog/v2.py · líneas 150–162

```
150 def submit_listing(**listing) -> dict:
151     """Deliver the listing. It is validated in code; if there are problems you get them back."""
152     candidate = merge_resolved(Listing.model_validate(listing), ev.worklist.resolved)
153     submissions.append(candidate)
154     problems = validate(candidate, schema, ev.category_urn)
155     if problems:
156         raise ValueError("The listing did not pass validation. Fix these and submit again:\n- " + "\n- ".join(problems))
157     return candidate.model_dump()
158
159 tools = [
160     FunctionTool.from_defaults(fn=lookup_corrections, name="lookup_corrections"),
161     FunctionTool.from_defaults(fn=submit_listing, name="submit_listing", fn_schema=Listing, return_direct=True,
162                               description="Deliver the final listing. It is validated in code."),
```

Cada entrega queda en submissions. Si se agotan las cinco, finalize limpia la última.

Resolver con las tablas lo que saben y controlar lo que decide el agente

Demo · la V2 sobre el mismo caso.

```
scripts/experiment.sh --version v2 --data mock --case 01-real-calota-ar014
```

- source, en la traza: con run_agent salió del agente; sin él, de la caché o de las tablas
- Los atributos que resolvió el código: la lista resolved del mensaje al agente
- Las entregas que rechazó submit_listing: cada llamada a la herramienta con su error
- Los tokens de la traza: nuevos, leídos de caché y escritos en caché

Respaldo · el experimento v2-mock del 1/10, abierto en Langfuse

Resolver con las tablas lo que saben y controlar lo que decide el agente

Costo por producto, medido.

Medido el 1/10 en Langfuse · 30 reales · promedio por producto

Versión	Entrada sin caché	Leída de caché	Segundos
V1	~4.560	~14.780	~3,3
V2	~9.970	0	~12,4

Resolver con las tablas lo que saben y controlar lo que decide el agente

¿Cuánto puede costar?

Abierta: el tope se acuerda con Alephee sobre la medición del 1/10

DECISIÓN 11

¿Cómo tratamos la restricción de costo del modelo?

A · Tope mensual acordado con Alephee y medición por producto: tokens de entrada, salida, caché leída y escrita

B · Sin tope: se mide después

C · El tope de hoy (USD 350) y publicar sin atributos al agotarse

PROPUESTA · A. Hoy el tope corta la calidad, que es la opción C. Medido el 1/10, la V2 paga unos 9.970 tokens de entrada nuevos por producto y no leyó caché, contra 4.560 nuevos y 14.780 leídos de caché de la V1. Con la caché por SKU (decisión 13), el mismo producto que venden 40 concesionarios se paga una vez. El tope en dólares se fija con Alephee sobre esa medición, después de revisar por qué la V2 no leyó caché.

decisiones/11-costo.md

Resolver con las tablas lo que saben y controlar lo que decide el agente

Guardrail: una comprobación en código.

- **VALIDATE** | Lista los problemas en lenguaje claro. `submit_listing` se los devuelve al agente, que corrige y vuelve a entregar, hasta cinco veces
- **CLEAN** | La red final: fija la categoría de la tabla, descarta lo inválido con su motivo y marca los obligatorios que faltan
- **NUNCA PUBLICA** | Un valor fuera de la lista del canal, un -1 ni un duplicado. No verifica que el valor sea cierto: eso depende del esquema y de catálogo

Resolver con las tablas lo que saben y controlar lo que decide el agente

La validación, en código.

core/src/catalog/guardrails.py · líneas 45–60

```
45 def validate(listing: Listing, schema: dict, category_urn: str) -> list[str]:
46     """Every contract problem of a listing, as messages for the agent. Empty list means valid."""
47     definitions = _definitions(schema)
48     problems = []
49     # The reference table decides the category; the model cannot change it.
50     if listing.category != category_urn:
51         problems.append(f"category must be {category_urn} (it comes from the reference table)")
52     problems += [p for a in listing.attributes if (p := _attribute_problem(a, definitions, category_urn))]
53     counts = Counter(a.urn for a in listing.attributes)
54     problems += [f"{urn} appears {n} times; keep only one" for urn, n in counts.items() if n > 1]
55     # A mandatory attribute must be either filled or declared missing with a reason. Silently
56     # leaving it out is what today's process does when the budget runs out.
57     missing = {m.urn for m in listing.missing}
58     problems += [f"{urn} ({d.get('name')}) is mandatory: fill it or add it to missing with the reason"
59                 for urn, d in definitions.items() if d.get("mandatory") and urn not in counts and urn not in missing]
60     return problems
```

Devuelve texto para el modelo. Una lista vacía quiere decir que la entrega pasa.

Resolver con las tablas lo que saben y controlar lo que decide el agente

¿Qué hace cuando no sabe?

La V2 se construyó con la opción B · C se puede armar encima

DECISIÓN 12

¿Qué hace el agente cuando no puede completar un atributo obligatorio?

A · Publicar sin el atributo (hoy)

B · Faltante explícito con motivo: la publicación sale con missing y alguien decide

C · Bloquear la publicación hasta revisión humana

PROPUESTA · B. Reemplaza el publicar sin atributos. En la V2, validate rechaza una entrega con un obligatorio sin valor ni motivo, y clean lo marca en missing si el agente no lo corrige. Sin categoría en la tabla o sin esquema, el Workflow termina sin llamar al modelo, con missing: category. Quién recibe la lista missing y si bloquea la publicación es parte de la integración con la plataforma de Alephee, y queda pendiente con dueño. C es una política válida que se puede construir sobre B.

decisiones/12-cuando-no-sabe.md

Resolver con las tablas lo que saben y controlar lo que decide el agente

Memoria: correcciones del equipo de catálogo.

- Una corrección dice: en esta categoría, este valor del producto va a este atributo con este valor del canal
- El agente la consulta con `lookup_corrections`, y la corrección manda sobre su criterio
- En la V2 vive en DynamoDB (tabla `corrections`) y se carga con `scripts/correct.sh`. En producción, una tabla de Alephee o AgentCore Memory

Resolver con las tablas lo que saben y controlar lo que decide el agente

La caché: la clave y la lectura.

core/src/catalog/v2.py · líneas 95–112

```
95     sku = str(product.get("sku") or "").strip()
96     categories = product.get("categories") or []
97     if not sku or not categories:
98         return None
99     return (sku, legacy_id(categories[0]["urn"]))
100
101 @step
102 async def check_cache(self, ev: MappingRequested) -> MappingCompleted | CacheMissed:
103     """Return the cached listing if there is one and it still passes validation."""
104     key = self._cache_key(ev.product)
105     category = self.reference.category_for(ev.product)
106     schema = self.schemas.get(category["urn"]) if category else None
107     if key and schema:
108         hit = _safe(lambda: self.cache.get(*key, self.reference.version, self.prompt_version), None, "cache get")
109         # Re-validated against the current schema: a cached listing that no longer fits the
110         # channel (schema changed, bad entry) is ignored and the product is mapped again.
111         if hit is not None and not validate(hit, schema, category["urn"]):
112             return MappingCompleted(listing=hit, source="cache")
```

Un error de DynamoDB no frena el mapeo: `_safe` sigue sin caché.

Resolver con las tablas lo que saben y controlar lo que decide el agente

finalize: gana lo resuelto y se guarda lo válido.

core/src/catalog/v2.py · líneas 198–215

```
198 async def finalize(self, ev: AgentDone) -> MappingCompleted:
199     """Merge, clean and, only if the agent's listing was valid, store it in the cache."""
200     if ev.last is None:
201         return MappingCompleted(listing=None, error=ev.error or "the agent never submitted a listing", source="agent")
202     schema = self.schemas[ev.category_urn]
203     # Last safety net: whatever the agent left, the output is forced back into contract
204     # (category from the table, invalid values dropped, mandatory gaps listed in `missing`).
205     final = clean(merge_resolved(ev.last, ev.worklist.resolved), schema, ev.category_urn)
206     key = self._cache_key(ev.product)
207     # A listing the guardrails had to clean is not cached: the next run gets another chance
208     # instead of repeating a degraded answer for every dealer that sells the SKU.
209     if ev.valid and key:
210         _safe(lambda: self.cache.put(*key, self.reference.version, self.prompt_version, ev.category_urn, final),
211              None, "cache put")
212     # Never hide an agent error behind a cleaned listing; an exhausted loop with no other
213     # error gets a specific reason instead of silently looking like a clean success.
214     error = ev.error
215     if error is None and ev.exhausted and not ev.valid:
```

Sin ninguna entrega, el resultado lleva un error.

Resolver con las tablas lo que saben y controlar lo que decide el agente

¿Cómo garantizamos determinismo?

La V2 se construyó con la opción A · falta separar por cuenta

DECISIÓN 13

¿Cómo garantizamos que el mismo SKU dé la misma salida?

A · Caché por SKU + tablas + prompt en DynamoDB; se invalida al cargar una corrección

B · Seed fijo (lo que se intentó hoy; los parámetros exactos siguen sin confirmar)

C · Recalcular siempre y aceptar variación

PROPUESTA · A. Un mapeo por SKU, con la versión de las tablas y la del prompt en la clave, en la tabla mapping_cache de DynamoDB. Cambiar una tabla o el prompt cambia la clave. Cargar una corrección con scripts/correct.sh borra la caché de esa categoría. Solo se guarda una entrega que pasó la validación. B reduce la variación pero no la elimina, y sigue pagando cada corrida. Antes de producción falta separar por cuenta y sumar la versión del esquema del canal.

decisiones/13-determinismo-y-cache.md

Resolver con las tablas lo que saben y controlar lo que decide el agente

Demo · corregir y repetir.

```
scripts/correct.sh --category urn:category:102529:vendor:shopee --attribute  
urn:attribute:990001:vendor:shopee --product-value 14 --value-id 99102 --value '14''
```

- Primera corrida, `scripts/experiment.sh --version v2 --data mock --case 01-real-calota-aro14`: en la traza, `lookup_corrections` devuelve la corrección
- El valor de Aro sale tal cual lo cargó catálogo
- Segunda corrida, el mismo comando: sale de la caché (`source = cache`), sin `run_agent` ni llamadas al modelo
- `rejected` explica cada descarte de clean

Respaldo · las corridas del experimento v2-mock del 1/10, en Langfuse

Resolver con las tablas lo que saben y controlar lo que decide el agente

Demo · la V2 desde el chat.

```
API_URL=<Function URL del BFF> CHAT_HMAC_SECRET=<secreto HMAC> npm start -w apps/web
```

- Escribir en el chat: Mapea el SKU 94701411 con la V2
- El asistente llama a map_product_v2 y muestra la categoría, los atributos, los faltantes y source
- Pedirlo otra vez: source dice cache
- En Langfuse: la traza del turno del chat, con map_product_v2 y, adentro, los steps de la V2

Respaldo · npm run dev en localhost:3000: modo mock del chat, con una respuesta de ejemplo que no corre la V2

Resolver con las tablas lo que saben y controlar lo que decide el agente

Cómo lo probamos.

- Tests unitarios: `uv run pytest core/tests -q`. Son 181, con dobles del modelo, sin AWS ni Langfuse
- Experimentos en Langfuse: `scripts/experiment.sh --version v2 --data mock`. Cada producto queda con su traza y sus scores
- Tests e2e: `API_URL=<Function URL del BFF> scripts/e2e.sh`, contra Bedrock, DynamoDB y el chat desplegado. Sin `API_URL` se omite el test del chat. Los 14 pasaron el 1/10

Resolver con las tablas lo que saben y controlar lo que decide el agente

Un test e2e, en código.

core/tests/e2e/test_e2e_mapping.py · líneas 47–55

```
47 @pytest.mark.parametrize("case_id", sorted(CASES))
48 async def test_v2_never_delivers_invalid_listings(case_id, stores):
49     done = await _v2(stores, cache=InMemoryCache()).run(product=CASES[case_id]["product"])
50     if case_id == "09-sin-categoria":
51         assert done.source == "tables" and done.listing.missing[0].urn == "category"
52         return
53     assert done.listing is not None, done.error
54     if done.listing.category:
55         assert validate(done.listing, SCHEMAS[done.listing.category], done.listing.category) == []
```

Corre con scripts/e2e.sh, que pone RUN_E2E=1. Sin eso, se omite.

Medir contra el criterio y repartir lo que sigue

La prueba.

El proceso de hoy contra la V1 y la V2, sobre los 30 reales, con el criterio de la decisión 8.

Medir contra el criterio y repartir lo que sigue

Resultados del 1/10.

30 reales · salida esperada MOCK · Hoy: lo que publicó Alephee · V1 y V2: experimentos de Langfuse del 1/10

Métrica	Hoy	V1	V2
Casos exactos	7	3	5
Categoría correcta	28	28	27
Valores inválidos (-1 o fuera de lista)	47	3	0
Duplicados	5	0	0
Obligatorios sin informar	2	1	1
Precisión	0,70	0,70	0,69
Recall	0,97	0,57	0,60

Medir contra el criterio y repartir lo que sigue

Cómo leer la tabla.

- Exactos bajos en todas las columnas: la salida esperada hereda las omisiones del proceso actual y castiga aciertos que hoy nadie mapea
- Inválidos y duplicados sí son errores seguros. La V2 es la única columna con cero en los dos
- El obligatorio sin informar de la V2 (error-26301167) es Manufacturer (MOCK) = GM, tomado de Marca: la salida esperada mock no lo prevé
- Categoría: la V1 elige entre 23 opciones con la correcta adentro, así que su número es optimista

Medir contra el criterio y repartir lo que sigue

Camino a producción.

01 Esquema oficial de Shopee y salida esperada validada con el equipo de catálogo

02 Integración: la API pública lee el producto (F8), pero escribir publicaciones y leer tablas necesita acceso interno

03 Decidir dónde corre el batch (decisión 5) y qué pasa con missing (decisión 12)

04 Fijar el tope de costo con la medición del 1/10 (decisión 11) y probar Haiku 4.5 (decisión 6)

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (1 a 7).

N	Decisión	Archivo
1	Alcance: un canal, una familia, categoría + atributos	decisiones/01-alcance.md
2	Contrato: missing y rejected con motivo	decisiones/02-contrato.md
3	Tipo de aplicación: una llamada en la V1, Workflow con FunctionAgent en la V2	decisiones/03-tipo-de-aplicacion.md
4	Salida estructurada con Pydantic (Listing)	decisiones/04-salida-estructurada.md
5	Dónde corre: local hoy, chat en AgentCore, batch a medir	decisiones/05-donde-corre.md
6	Modelo: Claude Sonnet 5; Haiku 4.5 a probar	decisiones/06-modelo.md
7	Stack: Python + LlamaIndex Workflows + BedrockConverse + Langfuse	decisiones/07-stack.md

Medir contra el criterio y repartir lo que sigue

Las 13 decisiones (8 a 13).

N	Decisión	Archivo
8	Criterio de éxito: el número de la pizarra	decisiones/08-criterio-de-exito.md
9	Dataset: 30 reales con mock rotulado	decisiones/09-dataset.md
10	La tabla manda: categoría y campos en código; el agente decide valores y lo no cubierto	decisiones/10-tabla-vs-agente.md
11	Costo: tope acordado con Alephee sobre la medición del 1/10 (V2: ~9.970 tokens nuevos (sin caché) de entrada por producto)	decisiones/11-costo.md
12	Cuando no sabe: faltante explícito con motivo; clean es la red final	decisiones/12-cuando-no-sabe.md
13	Determinismo: caché por SKU + tablas + prompt en DynamoDB; se invalida al cargar una corrección	decisiones/13-determinismo-y-cache.md

Medir contra el criterio y repartir lo que sigue

Quién hace qué, para cuándo.

Se completa en la sala.